

Final Assignment
MAST32001 - Computational Statistics

Computational Statistics Final Report Group B

Hans Falkenberg, Andrea Magliani, Roghayyeh Rezvani, Henry Vaga

October 25, 2026

https://github.com/Roghayyeh2025/MAST32001-Computational-Statistics.git

Contents

1 Time series bootstrap and the ladle estimator **2**
1.1 Whitening and autocovariance matrices . . . 3
1.2 Bootstrap for dependent observations . . . 4
1.3 The ladle quantities . . . 6
1.4 Monte Carlo Simulation study with setting 1 . . . 8
1.5 Application to times-series data . . . 10

2 Bayesian mixture modelling with MCMC **13**
2.1 Latent-variable representation and Gibbs sampler . . . 13
2.2 Simulation study and computational behaviour . . . 16
2.3 Label switching and identifiability . . . 21
2.4 Application to real data . . . 23

1. Time series bootstrap and the ladle estimator

The aim of Task 1 is to investigate the estimation of the signal dimension in multivariate time series, following the framework presented by Nordhausen and Virta (2018)[†]. In particular, we focus on identifying how many latent components contain relevant temporal dependence, distinguishing them from components that behave as white noise. We consider the linear mixing model underlying Second-Order Source Separation (SOS), specifically, the signal-noise setting, where the components are divided into q signal components exhibiting temporal dependence and $p - q$ noise components behaving as white noise. The objective is to estimate the unknown signal dimension q from the observed data.

To investigate this problem, we use Second-Order Blind Identification (SOBI), which exploits the autocovariance structure of the time series at different lags. After centering and whitening the observations, SOBI jointly diagonalizes a collection of lagged autocovariance matrices to identify the latent directions associated with temporal dependence. However, SOBI alone does not directly determine how many components should be classified as signal.

For this purpose, we apply the ladle estimator, which combines information from the ordered SOBI eigenvalues with a measure of the stability of the estimated signal subspaces under bootstrap resampling. Since the observations are serially dependent, conventional independent and identically distributed bootstrap resampling may fail to preserve their temporal structure. We therefore investigate two alternatives: the fixed block bootstrap and the stationary bootstrap. Both methods resample consecutive observations in blocks, allowing temporal dependence to be retained within the resampled series.

The analysis is organized into five parts. First, we implement the centering, whitening, and lagged autocovariance calculations required for SOBI, and we verify the numerical properties of the whitening transformation. Next, we implement and investigate the two bootstrap procedures for dependent observations. We then construct the ladle estimator, examine its components, and we assess the performance of the procedure and the influence of the bootstrap method. Until now we work exclusively with simulated datasets based on Setting 1 of Nordhausen and Virta (2018). This allows us to evaluate the performance of the ladle estimator in a controlled setting, where the true signal dimension is known in advance ($q = 5$).

Each dataset consists of ten independent time series. The first five are temporally dependent signal components, generated according to the ARMA, AR and MA processes specified in Setting 1, while the remaining five are independent Gaussian white noise components. The first five series are scaled by their theoretical standard deviations to ensure unit variance, as required by the SOS model assumptions. Since the processes have theoretical mean zero, no sample mean subtraction is performed at this stage.

Finally, we apply both bootstrap-based estimators to a real multivariate time series, where the true signal dimension is unknown, comparing the results and investigating their sensitivity to the block length. Overall, the task aims to assess whether the combination of SOBI and bootstrap-based ladle estimation can provide a reliable and interpretable estimate of the number of temporally dependent latent components in multivariate time series.

[†]Nordhausen, K. and Virta, J. (2018). Ladle Estimator for Time Series Signal Dimension. In 2018 IEEE Statistical Signal Processing Workshop (SSP), pp. 428–432. DOI: 10.1109/SSP.2018.8450695.

1.1 Whitening and autocovariance matrices

In this paper we will be working with multivariate time series data $x_t \in \mathbb{R}^p$ for $t = 1, ..., n$. We will apply appropriate standardization of the data in accordance with the scientific paper*. To do this we are required to center and whiten data

$$x_t^{st} = \Sigma^{-1/2}(x - \mu) = \Sigma^{-1/2}x_c$$

with

$$\frac{1}{n-1}(x^c)^\top x^c = \Sigma \in \mathbb{R}^{p\times p}$$

being the sample covariance matrix of the centered data. We normalize with $\frac{1}{n-1}$ to unbias our covariance, since estimating the mean takes away one degree of freedom. While

$$\frac{1}{n}\sum_{t=1}^{n} x_t = \mu \in \mathbb{R}^p$$

is the sample mean of the original data.

When centering multivariate data, such as x, we want to stress that we are subtracting the column mean of each variable from the corresponding observations of that variable. We whiten the centered data using ZCA whitening matrix. For this, we will first do a Singular Value Decomposition (SVD) on the covariance matrix which is symmetric and positive semidefinite so

$$\Sigma = UDV^\top = Q\Lambda Q^\top$$
$$\Sigma^{-\frac{1}{2}} = Q\,\mathrm{diag}(\Lambda)^{-1/2}Q^\top.$$

This works as Q is orthogonal and Λ is a diagonal matrix. The decomposition reduces computation of finding the inverse. $\Sigma^{-\frac{1}{2}}$ is then the whitening matrix. To understand the benefit of whitening, we first recall the linear mixing model underlying Second-Order Source Separation (SOS), which forms the basis of the SOBI method. The observed multivariate time series can therefore be represented as

$$x_t = \mu + \Omega z_t$$

with $E(z_t) = 0$, and $cov(z_t) = \mathbf{I}_p$. We first decompose $\Omega = A\Lambda B^\top$. Hence,

$$cov(x_t) = \Omega\Omega^\top = A\Lambda B^\top B\Lambda A^\top = A\Lambda^2 A^\top = \Sigma$$

as A and B are orthonormal by definition of the Singular Value Decomposition (SVD), and all are matrices in $\mathbb{R}^{p\times p}$. *To build intuition, take $p = 2$ and $z_t \sim N(0, \mathbf{I}_2)$, so a sample of z_t forms a round cloud. B would then rotate the cloud leaving the shape unchanged, Λ would stretch it into an ellipse, and A would change the direction of the ellipse.*

Writing out the whitening matrix gives us $\Sigma^{-1/2} = A\Lambda^{-1}A^\top$ and whitening x_t gives

$$x_t^{st} = \Sigma^{-1/2}(x_t - \mu) = A\Lambda^{-1}A^\top A\Lambda B^\top z_t = AB^\top z_t.$$

This shows that whitening therefore removes any stretching Λ leaving only the unknown rotation of the orthogonal matrix $AB^\top$, which has fewer unknown variables than Ω. *Geometrically for our intuition this would mean that our whitened data is back to a unit circle, but the rotations of A and B still remain.* We verify that our whitening works on the simulated data from setting 1. Note that it is expected to have a numerical error at $e-16$ on a 64-bit computer, hence the off-diagonals are practically 0. Since the covariance matrix of the whitened data is the identity, our whitening has been successful as is shown in Table 1.1.

*Nordhausen, K. and Virta, J. (2018). Ladle Estimator for Time Series Signal Dimension. In 2018 IEEE Statistical Signal Processing Workshop (SSP), pp. 428–432. DOI: 10.1109/SSP.2018.8450695.

Table 1.1: Sample covariance of the whitened `setting1` data.

	x_1^{st}	x_2^{st}	$\cdots$	x_{10}^{st}
x_1^{st}	1	$-6.05e-16$	$\cdots$	$1.31e-16$
x_2^{st}	$-6.05e-16$	1	$\cdots$	$-3.49e-16$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
x_{10}^{st}	$1.31e-16$	$-3.49e-16$	$\cdots$	1

We will implement the autocovariance Σ_τ with different lags τ. This checks for any linear dependency in x between time point i and $i+\tau$. For the implementation of the sample Σ_τ we used the function

$$\Sigma_\tau = \frac{1}{n-\tau}\sum_{t=1}^{n-\tau}(x_t^{st}-\mu^{st})(x_{t+\tau}^{st}-\mu^{st})^\top = \frac{1}{n-\tau}\sum_{t=1}^{n-\tau}x_t^{st}x_{t+\tau}^{st\top}.$$

Conventions we have chosen for the autocovariance are:

- Subtracting the whole mean from both x_i and $x_{i+\tau}$, instead of subtracting the mean from the data available to X_i and $x_{i+\tau}$. This will matter little for our large time series and moderate number of lags, and we gain the simple formula for AC, as $\mu^{st}=0$, when working with the standardized data.
- Normalizing by $n-\tau$, which is the number of pairings we can make, yields the true average, and avoids the increasing bias arising from normalizing with n.
- We are interested in the lags from 1 to 12. Since 0 lag in our case is the covariance matrix which by construction of the whitened data is $\mathbf{I_p}$, it does not contain any information.

In R, we used the function `crossprod()` to compute the lagged cross-products efficiently, avoiding explicit transposition and taking advantage of optimized matrix multiplication routines.

1.2 Bootstrap for dependent observations

The bootstrap method must be chosen properly while working on serially dependent observations, as in the time series. The bootstrap should preserve the temporal dependence structure, so it can represent the true data. We therefore cannot use i.i.d. bootstrap, as it completely breaks the dependency of the observations. The fixed block bootstrap is a good alternative for preserving dependency.

The fixed block bootstrap does not resample single observations, it resamples blocks of observations of length l, where l is the tuning parameter that controls the block length. If number of samples is divisible by l, each block contains the same number of consecutive observations, otherwise the last block contains less consecutive observations.

Another option is the stationary bootstrap*, which also resamples blocks of observations, but the lengths of the blocks may differ. In this case, the length of a single block follows a geometric distribution Geom(p), where p is the tuning parameter that determines the expected block length $1/p$. With this bootstrap we get a stationary resampled time series, where the joint distribution of the series is invariant under time shifts, so its probabilistic properties do not depend on the time at which they are observed. Since SOBI relies on lagged autocovariance matrices, it assumes second-order stationarity, meaning that autocovariances depend only on the time lag, and not on the time. The stationary bootstrap preserves stationarity in the resampled process, making it particularly compatible with this assumption. Although, the fixed block bootstrap can also be valid, even if it does not give a stationary distribution.

The implementation of the fixed block bootstrap consists of creating a function that takes the multivariate data matrix and the tuning parameter l as the inputs, and gives the resampled data as the output. It will sample $m = \lceil \frac{n}{l} \rceil$ blocks of length l and at the end it will keep just the first n observations, since $m \cdot l$ can be greater than n. We did not choose to use a circular block bootstrap, so for every block the function samples an observation uniformly that stays in the position k between 1 and $n-l+1$, so that we can get blocks of length l. Then we add the observation in the positions $k, \ldots, k+l-1$ to the new resampled dataset.

*Politis, D. N. and Romano, J. P. (1994). The Stationary Bootstrap. Journal of the American Statistical Association, 89(428), 1303–1313. DOI: 10.1080/01621459.1994.10476870.

The stationary bootstrap takes as input the multivariate data matrix and the tuning parameter p. In this case, the length of the blocks are not fixed numbers, but are identically distributed geometric random variables with parameter p. We have $L \sim \text{Geom}(p)$, so $P(L = l) = (1-p)^{l-1} \cdot p$, and the expected block length is $1/p$, which is the expected value of the geometric distribution. So, we have that at the start of each block where an observation gets picked by a uniform distribution on a range from 1 to n. In the stationary bootstrap, blocks are constructed using circular indexing, meaning that a block can start at the last observation and continue from the first observation of the original time series.

To implement this, we worked with an extended dataset containing $2 \cdot n$ rows, obtained by concatenating the original dataset with itself. Thus, rows $1, \ldots, n$ contain the original observations, while rows $n+1, \ldots, 2n$ repeat the same observations in the same order. This allows blocks to wrap around the end of the original series without requiring explicit circular indexing. So an observation gets sampled to start a block and then we have two options. Either with probability $(1-p)$ the block continues, and the observation in the position after the current one gets added to the block and so to the resampled data, or with probability p the block finishes, and we have to uniformly sample a new observation to start a new block. In R we use `runif(1)` to decide whether we continue with the same block or not. If `runif(1)>=p`, we continue, and otherwise if `runif(1)<p`, we start a new block. Due to the random nature of the geometric distribution, block lengths vary from one block to another.

In both cases, either fixed block or stationary bootstrap, the choice of the tuning parameter is important and requires particular attention, since a small length of the blocks gives us new data where the dependence is not well preserved. This is the problem with the i.i.d. bootstrap, which completely changes the temporal structure of the data. So a larger length helps us keep longer blocks of observations together, and also some temporal dependency, but if it is too large, we have too few blocks sampled, and the bootstrap variability is unstable. If we have 1000 observations we will avoid using block lengths of 5 or 500, but finding the right length, or expected length, is a tough job that does not have an universal answer and requires some diagnostics.

To ease notation, we use the term block length l for both bootstrap methods. For the fixed bootstrap this is the length of every block, while for the stationary bootstrap it is the expected block length, so that each new observation starts a new block with probability $1/l$. For example, $l = 20$ for the stationary bootstrap means an expected block length of 20, that is, a probability of $1/20$. We work with a simulation of `setting1` to see how some different bootstraps behave in different cases.

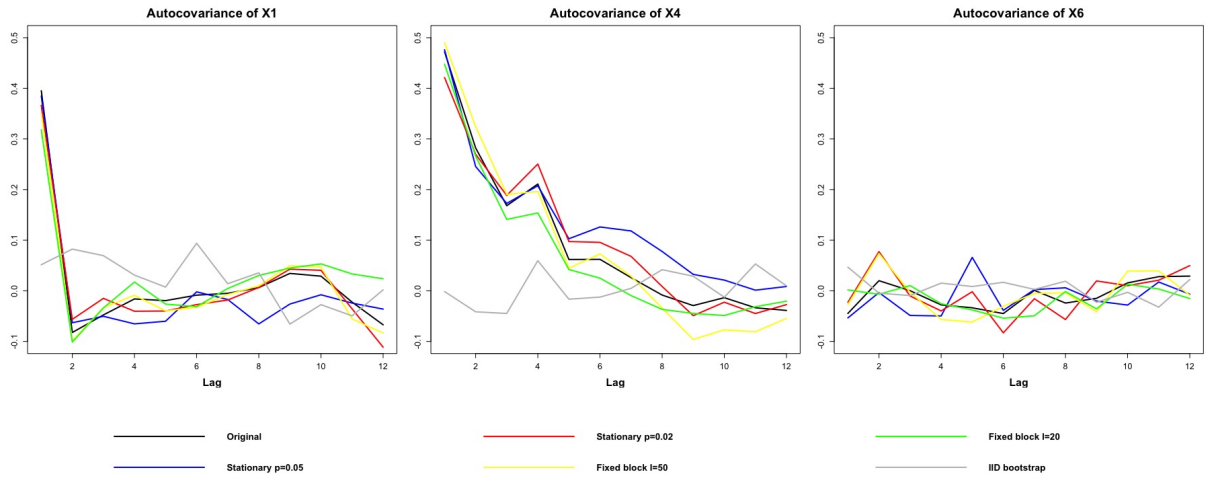


Figure 1.1: Comparison of lagged autocovariances for X_1, X_4 and X_6 under different bootstrap methods. The original series is compared with i.i.d., stationary ($p = 0.05$ and $p = 0.02$) and fixed block ($l = 20$ and $l = 50$) bootstrap samples.

In Figure 1.1 we compare how the fixed and stationary bootstrap, with $l = 20$ and $l = 50$, and the i.i.d. bootstrap behave for three different situations: the ARMA variable X_1, the MA variable X_4, and a Gaussian white noise X_6. For X_1, we have that the original observations present a strong dependence, which is partly represented by the stationary and fixed block bootstraps. In particular the cases $l = 50$ give us a close result to the original one. It shows that bootstraps with longer blocks preserve more of

the temporal dependence of the observations. In this case as we expected, the i.i.d. bootstrap completely breaks the structure of the time series, and it does the same for X_4.

For X_4, the stationary and fixed bootstrap with $l = 50$ appear to reproduce the original autocovariances more closely at the first few lags. However, as the lag increases, both bootstraps with $l = 20$ seem to provide a closer approximation to the original curve. Nevertheless, since X_4 is an MA(4) process, its theoretical autocovariances are zero for lags greater than 4. Therefore, the deviations of the original sample autocovariances from zero at higher lags are due to sampling variability, and a closer fit to the original curve does not necessarily imply better performance.

For X_6, all bootstrap methods produce autocovariances fluctuating around zero, with relatively small differences between them. This is consistent with the fact that X_6 is Gaussian white noise, whose theoretical autocovariances are zero at all positive lags. Although, the i.i.d. bootstrap is theoretically well suited to this process because it preserves independence, the plot does not show a clear advantage over the block bootstrap methods.

1.3 The ladle quantities

The aim of the ladle estimator is to estimate the unknown signal dimension q, distinguishing the components containing temporal dependence from those corresponding to white noise. The method combines two sources of information: the eigenvalues, which reflect the strength of the temporal dependence, and the eigenvectors, whose stability is assessed through bootstrap resampling. By combining these two pieces of information, the ladle estimator provides a criterion for selecting the signal dimension. The estimator is based on the function $g(k) = f(k) + \phi(k)$, where $f(k)$ measures the bootstrap instability of the estimated subspace and $\phi(k)$ is constructed from the ordered eigenvalues. The estimated signal dimension is then obtained by minimizing $g(k)$ over the candidate dimensions.

To get the estimate, first we whiten the data and obtain 12 autocovariance matrices of dimension $p \times p$, corresponding to the 12 selected time lags. These matrices are then approximately jointly diagonalized, since SOBI assumes that the latent signals are temporally uncorrelated with each other. Joint diagonalization aims to recover the latent components by finding a transformation that makes the autocovariance matrices as diagonal as possible across all considered lags. The joint diagonalization gives us U, a $p \times p$ diagonalizer, and an array of 12 approximately diagonal matrices. Indeed, approximately 25% of the total mass of the 12 transformed autocovariance matrices remains outside their diagonals. The theoretical autocovariance matrices are jointly diagonalizable under the SOBI assumptions, but their sample estimates are generally not exactly simultaneously diagonalizable. For each component i, we compute

$$\lambda_i = \sum_{\tau=1}^{12} (D_\tau)_{ii}^2,$$

where D_τ denotes the diagonalized autocovariance matrix at lag τ. The resulting values are ordered such that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{10}$, so using the simulation setting `Setting1` we get

$$\lambda = (2.441 \quad 0.437 \quad 0.205 \quad 0.182 \quad 0.159 \quad 0.023 \quad 0.022 \quad 0.021 \quad 0.0122 \quad 0.009).$$

Each λ_i measures the overall strength of the temporal dependence of the corresponding variable across the selected lags. Signal components are expected to have positive values, whereas white noise components have theoretically zero autocovariances at positive lags and therefore zero values of λ_i. In finite samples, however, the values associated with noise components are generally small rather than exactly zero.

The eigenvalue-based component of the ladle estimator is defined as

$$\phi(k) = \frac{\lambda_{k+1}}{1 + \sum_{j=1}^{10} \lambda_j}, \qquad k = 0, \ldots, p-1,$$

we get

$$\phi(k) = (0.5437 \quad 0.0950 \quad 0.0456 \quad 0.0405 \quad 0.0332 \quad 0.0051 \quad 0.0048 \quad 0.0048 \quad 0.0027 \quad 0.0020).$$

This function measures the remaining temporal dependence after considering the first k components as signals. However, $\phi(k)$ alone is not sufficient to identify the exact signal dimension, since we should

identify an elbow in the eigenvalue curve to pick the final q, but the elbow may not be clearly defined, making its identification subjective and open to interpretation. On the other hand, the ladle estimator is not subjective; we just pick the q with the lowest g value.

This motivates the introduction of the eigenvector-based component $f(k)$, which measures the instability of the estimated subspaces through bootstrap resampling. The underlying idea is that identifiable signal subspaces should remain relatively stable, whereas subspaces containing only part of the non-identifiable noise space may be unstable. This is because the individual noise directions are not uniquely identifiable when their associated eigenvalues are equal, and they may become unstable when these eigenvalues are very close to each other. Therefore, $f(k)$ provides information about subspace stability, helping to identify the boundary between signal and noise components.

To look at that stability we used two stationary bootstrap and fixed block bootstrap with $l = 20$. In both cases, we got 500 different bootstraps of the original dataset, whitened every bootstrap, and then we did the joint diagonalization, that gave us the information necessary to get f. In this case, we are not interested in the diagonalized array of 12 $p \times p$ matrices, but on the orthogonal $p \times p$ diagonalizer U. $U^T U$ gives us almost $\mathbf{I}_p$, though not exactly $\mathbf{I}_p$ because of the floating point numerical precision of the computer, hence U is orthogonal.

For the original data and for every bootstrap, we create new matrices where the columns of the diagonalizer U are reordered according to the corresponding eigenvalues λ_i, arranged in decreasing order: $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_{10}$. Therefore, the first columns of U correspond to the components with the strongest temporal dependence, while the last columns correspond to components with weaker temporal dependence, potentially representing white noise. Let $\widehat{B}_k$ denote the matrix containing the first k columns of the joint diagonalizer estimated from the original data, and let $\widehat{B}_k^{*(j)}$ be the corresponding matrix obtained from the j-th bootstrap sample, for $j = 1, \ldots, B$.

The variability of the estimated k-dimensional subspace is measured by comparing the column spaces of these matrices. In particular, we define

$$f_0(k) = \frac{1}{B} \sum_{j=1}^{B} \left[1 - \left| \det \left(\widehat{B}_k^\top \widehat{B}_k^{*(j)} \right) \right| \right],$$

for $k = 1, \ldots, p-1$, with $f_0(0) = 0$. The determinant measures the similarity between the two estimated subspaces, therefore, larger values of $f_0(k)$ indicate greater instability of the estimated k-dimensional subspace. Finally, the normalized instability function is defined as

$$f(k) = \frac{f_0(k)}{1 + \sum_{\ell=0}^{p-1} f_0(\ell)}.$$

We normalize to make the contribution of subspace instability comparable to that of the eigenvalue-based component $\phi(k)$ in the ladle estimator. For the stationary bootstrap we got

$$f(k) = (0 \quad 0.0025 \quad 0.0157 \quad 0.0816 \quad 0.0399 \quad 0.0166 \quad 0.1449 \quad 0.1697 \quad 0.1615 \quad 0.1362),$$

while for the fixed block bootstrap we got

$$f(k) = (0 \quad 0.0025 \quad 0.0154 \quad 0.0738 \quad 0.0334 \quad 0.0173 \quad 0.1494 \quad 0.1748 \quad 0.1632 \quad 0.1378).$$

Figure 1.2 shows that the ladle plots obtained using both bootstrap methods on 500 bootstrap replications with $l = 20$, are remarkably similar. While the eigenvalue component $\phi(k)$ is identical in both cases, the bootstrap instability component $f(k)$ shows only minor differences. In fact, we have

$$g(k) = (0.5437 \quad 0.0975 \quad 0.0613 \quad 0.1220 \quad 0.0731 \quad 0.0216 \quad 0.1498 \quad 0.1745 \quad 0.1643 \quad 0.1382)$$

for the stationary bootstrap, and

$$g(k) = (0.5437 \quad 0.0975 \quad 0.0610 \quad 0.1143 \quad 0.0666 \quad 0.0223 \quad 0.1543 \quad 0.1796 \quad 0.1659 \quad 0.1397)$$

where $k = 0, \ldots, p-1$, for the fixed block bootstrap. Consequently, the ladle criterion $g(k)$ reaches its minimum at $k = 5$ for both methods, correctly identifying the true signal dimension. These results

suggest that, for this particular dataset, the estimated signal dimension is robust to the choice between the two bootstrap procedures. The eigenvalue component $\phi(k)$ and the bootstrap instability component $f(k)$ intersect approximately at $k = 4$, while the ladle criterion $g(k) = f(k)+\phi(k)$ reaches its minimum at $k = 5$, correctly identifying the true signal dimension. This behavior illustrates the trade-off between the two components: $\phi(k)$ decreases as more signal components are included, whereas $f(k)$ tends to increase when additional, less stable noise directions are considered. Although the intersection occurs close to the minimum, the two points do not necessarily coincide, since the ladle estimator selects the value of k that minimizes their sum rather than the point where the two components are equal.

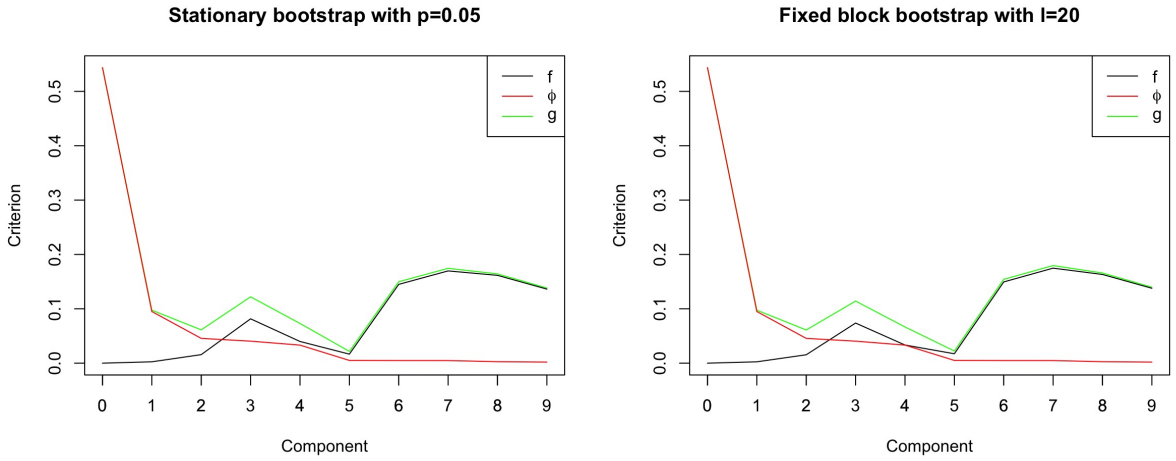


Figure 1.2: Comparison of the ladle estimator for Setting 1 ($n = 1000$) using stationary bootstrap ($p = 0.05$) and fixed block bootstrap ($l = 20$), with 500 bootstrap replications for each method.

1.4 Monte Carlo Simulation study with setting 1

We now have everything to perform an extensive Monte Carlo analysis for our ladle method. We have chosen to do the analysis on $R = 100$ Monte Carlo repetitions each done on a new draw from `setting1`. We justify $R = 100$ as follows. Since each replication either estimates q correctly or not, the indicator $\mathbf{1}\{\hat{q} = q\}$ is Bernoulli with success probability $\pi = P(\hat{q} = q)$. The accuracy is estimated by the proportion of correct replications $\hat{\pi}$, whose Monte Carlo standard error is

$$\text{MCSE}(\hat{\pi}) = \sqrt{\frac{\pi(1-\pi)}{R}}.$$

This is at most 0.05, attained at $\pi = 0.5$, and much smaller when π is close to 0 or 1. This is precise enough to detect the large differences in accuracy across sample sizes and block length. Being able to draw new samples allows us to test the performance of our method, as we have both the randomness from new draws and new bootstraps. Because the true dimension $q = 5$ is known, we can check how often $\hat{q}$ equals the true value and how much it varies between data sets. This can not be done when resampling from the same dataset. More bootstraps would only decrease the MCSE of the bootstrap but add no additional information beyond what the one sample already contains, hence it cannot reveal how $\hat{q}$ would change for new samples of the same process. We did not choose a higher R, in order to get a good compromise between computational efficiency and the precision of the estimate. Specifications and how it will allow us to reproduce the main qualitative behavior of the paper*:

- Bootstrap samples $B = 100$,
- Sample size $n \in \{200, 500, 1000, 1500\}$. This will allow us to reproduce what sample sizes the method fails or works with.
- Bootstrap method $m \in \{\text{stationary}, \text{fixed}\}$. Changing method is essential to check for differences between fixed and stationary bootstrap.

*Nordhausen, K. and Virta, J. (2018). Ladle Estimator for Time Series Signal Dimension. In 2018 IEEE Statistical Signal Processing Workshop (SSP), pp. 428–432. DOI: 10.1109/SSP.2018.8450695.

- Block length for bootstrap methods $l \in \{5, 10, 20, 30, 50\}$. By including many lengths we will be able to reproduce the effect of too short blocks and find the most suitable block length. We will keep the block lengths relatively close to n, such that we do not get unstable bootstrap variability. We chose not to consider $l = p = 1$ as this would lead to an i.i.d. draw, and therefore break the serial dependency. We clarify that methods and block lengths are run on the same 100 samples, to improve comparison between the settings.

Table 1.2: Distribution of $\hat{q}$ (in %) over $R = 100$ Monte Carlo replications. The last column gives the Monte Carlo standard error of the accuracy $\hat{\pi}$ (the $\hat{q} = 5$ column), in percentage points. An MCSE of 0 occurs when replications are either all correct or all wrong, and it does not mean there is no uncertainty.

n	Method	l	$\hat{q}=0$	$\hat{q}=1$	$\hat{q}=2$	$\hat{q}=3$	$\hat{q}=4$	$\hat{q}=5$	$\hat{q}=6$	$\hat{q}=7$	$\hat{q}=8$	$\hat{q}=9$	MCSE
200	Stationary	5	0	35	8	0	0	0	0	0	0	57	0.0
		10	0	42	12	2	0	0	0	0	0	44	0.0
		20	0	43	14	2	0	0	1	0	0	40	0.0
		30	0	44	16	4	1	0	1	0	0	34	0.0
		50	0	45	20	5	2	0	1	0	1	26	0.0
	Fixed	5	0	32	8	0	0	0	0	0	0	60	0.0
		10	0	42	14	3	0	0	0	0	0	41	0.0
		20	0	42	14	1	0	0	0	0	0	43	0.0
		30	0	42	15	6	0	0	0	0	0	37	0.0
		50	0	40	20	5	1	0	0	0	0	34	0.0
500	Stationary	5	0	48	18	3	9	22	0	0	0	0	4.1
		10	0	39	17	2	10	32	0	0	0	0	4.7
		20	0	38	15	3	8	36	0	0	0	0	4.8
		30	0	34	17	3	8	38	0	0	0	0	4.9
		50	0	32	15	3	7	43	0	0	0	0	5.0
	Fixed	5	0	51	19	2	12	14	0	0	0	2	3.5
		10	0	35	15	5	9	36	0	0	0	0	4.8
		20	0	40	17	2	6	35	0	0	0	0	4.8
		30	0	36	17	2	6	39	0	0	0	0	4.9
		50	0	33	19	3	6	39	0	0	0	0	4.9
1000	Stationary	5	0	0	2	0	18	80	0	0	0	0	4.0
		10	0	0	1	0	3	96	0	0	0	0	2.0
		20	0	0	0	0	2	98	0	0	0	0	1.4
		30	0	0	0	0	2	98	0	0	0	0	1.4
		50	0	0	0	0	2	98	0	0	0	0	1.4
	Fixed	5	0	1	4	1	30	64	0	0	0	0	4.8
		10	0	0	1	0	4	95	0	0	0	0	2.2
		20	0	0	0	0	2	98	0	0	0	0	1.4
		30	0	0	0	0	2	98	0	0	0	0	1.4
		50	0	0	0	0	2	98	0	0	0	0	1.4
1500	Stationary	5	0	0	0	0	7	93	0	0	0	0	2.6
		10	0	0	0	0	0	100	0	0	0	0	0.0
		20	0	0	0	0	0	100	0	0	0	0	0.0
		30	0	0	0	0	0	100	0	0	0	0	0.0
		50	0	0	0	0	0	100	0	0	0	0	0.0
	Fixed	5	0	0	0	0	13	87	0	0	0	0	3.4
		10	0	0	0	0	0	100	0	0	0	0	0.0
		20	0	0	0	0	0	100	0	0	0	0	0.0
		30	0	0	0	0	0	100	0	0	0	0	0.0
		50	0	0	0	0	0	100	0	0	0	0	0.0

As shown in Table 1.2, we first notice that both methods fail for any choice of tuning parameter for $n \in \{200, 500\}$, as $\hat{q} = q$ in only $0 - 43\%$ of the replicates, which does not suffice. For $n = 1500$ any method is good, when disregarding $l = 5$ as $\hat{q} = q$ in 100% of the draws.

For $n = 1000$, there is some variability in the distribution of $\hat{q}$s, and now both $l \in 5, 10$ seem worse than the other tuning parameters. This can be explained by the fact that SOBI used 12 lags, and a

block of length $5-10$ cannot contain this many lags. For $l=5$ the stationary bootstrap appears to do better with $\hat{\pi}_{s,0.2}=80\%$ versus $\hat{\pi}_{f,5}=64\%$ which is a difference of 0.16. To test the null hypothesis H_0: $\pi_{s,0.2}=\pi_{f,5}$, we compute the standard error of the difference

$$\mathrm{SE}(\hat{\pi}_S-\hat{\pi}_F)=\sqrt{\mathrm{MCSE}_S^2+\mathrm{MCSE}_F^2}=\sqrt{0.040^2+0.048^2}=\sqrt{0.0016+0.0023}\approx 0.063$$

and the corresponding normal 95% confidence interval $0.16\pm 1.96\cdot 0.063=[0.04,\ 0.28]$. As $0\notin[0.04,\ 0.28]$ we reject the null hypothesis at a 5% significance level. The difference could be explained by the fact that some of the blocks for the stationary bootstrap will have length above 12 even for $p=0.2$. At $n=1000$ we no longer get as strong results for any of the settings compared to $n=1500$, as $\hat{\pi}<1$, but the method still appears to work well for this data and sample size.

With reasonable parameters, 100 Monte Carlo replications is not enough to tell the fixed and stationary bootstrap apart. So now that we have done a preliminary analysis for many settings, where we have learned when our methods jointly fail or succeed, we are now going to investigate the two methods further by only considering $n=1000$ for $R=1000$, $B=100$, $l\in\{10,20,30\}$, and $p\in\{0.1,0.05,0.033\}$, to check for any difference between the two. Table 1.3 presents the test of H_0: $\pi_S=\pi_F$, where R is reduced to 1000 to lower computational costs.

Table 1.3: Test of $H_0:\pi_S=\pi_F$ at $n=1000$ with $R=1000$ replications.

l	$\hat{\pi}_F$	$\hat{\pi}_S$	$\hat{\pi}_S-\hat{\pi}_F$	95% CI	p-value
10	0.958	0.966	0.008	$[-0.009, 0.025]$	0.35
20	0.981	0.980	-0.001	$[-0.013, 0.011]$	0.87
30	0.986	0.982	-0.004	$[-0.015, 0.007]$	0.48

With $R=1000$, the accuracies of the two bootstrap methods differ by less than one percentage point for every block length, and none of the differences is significant at the 5% level. The confidence intervals show that any true difference is at most about 1 to 2.5 percentage points. Together with the result for $l=5$, this suggests that the bootstrap method only matters when the blocks are too short to contain the lags used by SOBI. It is also worth noting how the estimator fails, as pooled over both methods and all three block lengths, 92% of the incorrect estimates are $\hat{q}=4$ and the rest are $\hat{q}=2$. The dimension is thus never overestimated, and when it is wrong it is almost always one too small, which suggests that a signal with weak serial dependence is occasionally mistaken for noise.

For many reasons an analysis on simulated data with a known distribution can not necessarily be translated to real data. Firstly, we are not able to choose n freely for real datasets, but it is still useful to see for what sample sizes the method breaks and what expected block lengths are suitable.

Secondly, the simulation results describe the performance of the ladle under Setting 1, that is, stationary Gaussian ARMA signals with Gaussian white noise. Because SOBI is affine equivariant, the choice of mixing matrix does not matter, but the accuracy on real data also depends on the strength of the signals serial dependence, the type of noise, and stationarity. Our bootstrap requires stationarity, as it puts blocks together as if they were interchangeable, and SOBI also requires it to estimate the autocovariance.

If there were to be some trend β in data $x_t=\mu+\beta t+\Omega z_t$, the interchangeability would break, and we would not be able to estimate the autocovariance, since a trend would be registered as very strong serial dependency, leading to an inflated $\hat{q}$. Sometimes these trends can be removed, for example, by regressing the series on t, but this we will not go into detail about. Finally, we use SOBI with 12 lags, so our method would not apply as well, since we can not detect autocovariation beyond those lags, and we do not know what would happen if we were to increase SOBI beyond 12 lags.

1.5 Application to times-series data

We now apply the ladle estimator to the real time series dataset provided for the assignment. Unlike Setting 1, where the true signal dimension was known, the number of signal components in the real data is unknown and must be estimated. To this end, we use the same implementation developed and validated for Setting 1, applying both the stationary and fixed block bootstrap methods to estimate

the signal dimension with 500 bootstraps each. Furthermore, we are interested in the results obtained with different block lengths just to check for the stability of our results. We apply the ladle estimator implementation for the fixed block bootstrap and stationary bootstrap with length $10, 20, 30, 40$ and 50.

Table 1.4 reports the values of the ladle criterion $g(k)$, for $k = 0, \ldots, 9$, obtained using both stationary and fixed block bootstrap methods with different block lengths. The results show a clear dependence on the block length when relatively short blocks are used. In particular, for $l = 10$, both bootstrap methods select $\hat{q} = 2$, whereas for all block lengths between 20 and 50, both methods consistently select $\hat{q} = 3$. This suggests that the estimated signal dimension is robust to the choice of bootstrap procedure and relatively insensitive to the block length within the interval $20 \leq l \leq 50$, although some sensitivity is observed for shorter blocks.

A closer examination of the criterion values helps explain this behavior. For the stationary bootstrap with $l = 10$, the values $g(2) = 0.078$ and $g(3) = 0.080$ are extremely close, indicating that the distinction between two and three signal components is weak. Similarly, the fixed block bootstrap gives $g(2) = 0.081$ and $g(3) = 0.105$, favouring two components more clearly. However, as the block length increases, $g(2)$ remains approximately constant, generally between 0.074 and 0.076, while $g(3)$ decreases substantially. For example, under the stationary bootstrap, $g(3)$ decreases from approximately 0.080 at $l = 10$ to 0.038 at $l = 50$. The fixed block bootstrap exhibits a similar pattern, with $g(3)$ decreasing from approximately 0.105 to 0.039. Since the eigenvalue component $\phi(k)$ is computed from the original dataset and does not depend on the bootstrap procedure, the observed changes in $g(k)$ arise from the bootstrap instability component $f(k)$. In particular, the decrease in $g(3)$ indicates that the estimated three-dimensional signal subspace becomes more stable under the bootstrap as the block length increases.

One possible explanation is that longer blocks preserve more of the temporal dependence relevant to identifying the third signal component. However, this interpretation should be treated cautiously, since the observed differences may also reflect bootstrap Monte Carlo variability. Another important observation is the close agreement between the stationary and fixed block bootstrap methods. Although their criterion values are not identical, they produce the same estimated signal dimension for every block length considered. This suggests that, for these data, the choice of block length has a greater influence on the estimated dimension than the choice between the two bootstrap procedures.

Table 1.4: Sensitivity analysis of the ladle criterion for different bootstrap methods and block lengths.

Bootstrap method	$g(0)$	$g(1)$	$g(2)$	$g(3)$	$g(4)$	$g(5)$	$g(6)$	$g(7)$	$g(8)$	$g(9)$	$\hat{q}$
Stationary ($l = 10$)	0.195	0.179	0.078	0.080	0.129	0.143	0.149	0.148	0.139	0.114	2
Fixed block ($l = 10$)	0.195	0.175	0.081	0.105	0.132	0.141	0.145	0.143	0.133	0.113	2
Stationary ($l = 20$)	0.195	0.187	0.076	0.051	0.127	0.146	0.154	0.152	0.143	0.115	3
Fixed block ($l = 20$)	0.195	0.188	0.075	0.062	0.130	0.144	0.152	0.149	0.141	0.116	3
Stationary ($l = 30$)	0.195	0.191	0.075	0.044	0.127	0.145	0.155	0.154	0.144	0.116	3
Fixed block ($l = 30$)	0.195	0.202	0.075	0.051	0.129	0.143	0.149	0.151	0.141	0.115	3
Stationary ($l = 40$)	0.195	0.194	0.075	0.041	0.127	0.146	0.153	0.153	0.143	0.116	3
Fixed block ($l = 40$)	0.195	0.208	0.074	0.044	0.125	0.143	0.150	0.151	0.140	0.119	3
Stationary ($l = 50$)	0.195	0.195	0.076	0.038	0.127	0.142	0.154	0.155	0.145	0.116	3
Fixed block ($l = 50$)	0.195	0.217	0.074	0.039	0.125	0.142	0.151	0.150	0.139	0.119	3

Based on these findings, we select $l = 30$ for the final analysis. This represents an intermediate choice within the range of block lengths for which the estimated dimension is consistently equal to three. Moreover, at $l = 30$, the minimum at $k = 3$ is already clearly distinguished from the competing value at $k = 2$: the stationary bootstrap gives $g(3) = 0.044$ compared with $g(2) = 0.075$, while the fixed block bootstrap gives $g(3) = 0.051$ compared with $g(2) = 0.075$. Although $l = 50$ produces a more pronounced minimum of the ladle criterion, we choose $l = 30$ to allow greater resampling variability while still preserving temporal dependence. Overall, the sensitivity analysis provides evidence in favor of three signal components, while also highlighting the importance of selecting a sufficiently large block length. Nevertheless, since the true signal dimension is unknown for the real dataset, these results support the robustness of the estimate rather than establishing its correctness.

Figure 1.3 presents the ladle criterion $g(k)$ and its components, $f(k)$ and $\phi(k)$, for the real time series dataset, comparing the stationary bootstrap and the fixed block bootstrap both with block length 30. Both methods use 500 bootstrap replications. The most striking feature of the figure is the remarkable

similarity between the results obtained with the two bootstrap procedures. The curves of the overall ladle criterion $g(k)$ are almost overlapping, indicating that the choice of resampling method has very little influence on the results of $\hat{q}$. As expected, the eigenvalue-based component $\phi(k)$ is identical for both methods, since it is computed from the original data. The instability components $f(k)$ also follow very similar patterns, with only minor differences attributable to the different resampling mechanisms. Consequently, both bootstrap procedures lead to the same estimated signal dimension, $\hat{q} = 3$.

Another interesting feature is that the curves of $f(k)$ and $\phi(k)$ intersect almost exactly at $k = 3$. This suggests that, around the estimated signal dimension, the two components make comparable contributions to the overall criterion. For smaller values of k, the eigenvalue-based term is relatively more influential, while beyond the intersection the instability component becomes more prominent. This behavior is consistent with the idea that including additional components beyond the main signal subspace introduces directions that are less reliably identified across bootstrap samples. Nevertheless, the intersection should not be interpreted as the formal criterion for selecting the signal dimension, which is determined exclusively by minimizing $g(k) = f(k) + \phi(k)$. Indeed, the minimum of the ladle criterion occurs at $k = 3$ for both methods, with $g(3) \approx 0.044$ for the stationary bootstrap and $g(3) \approx 0.051$ for the fixed block bootstrap.

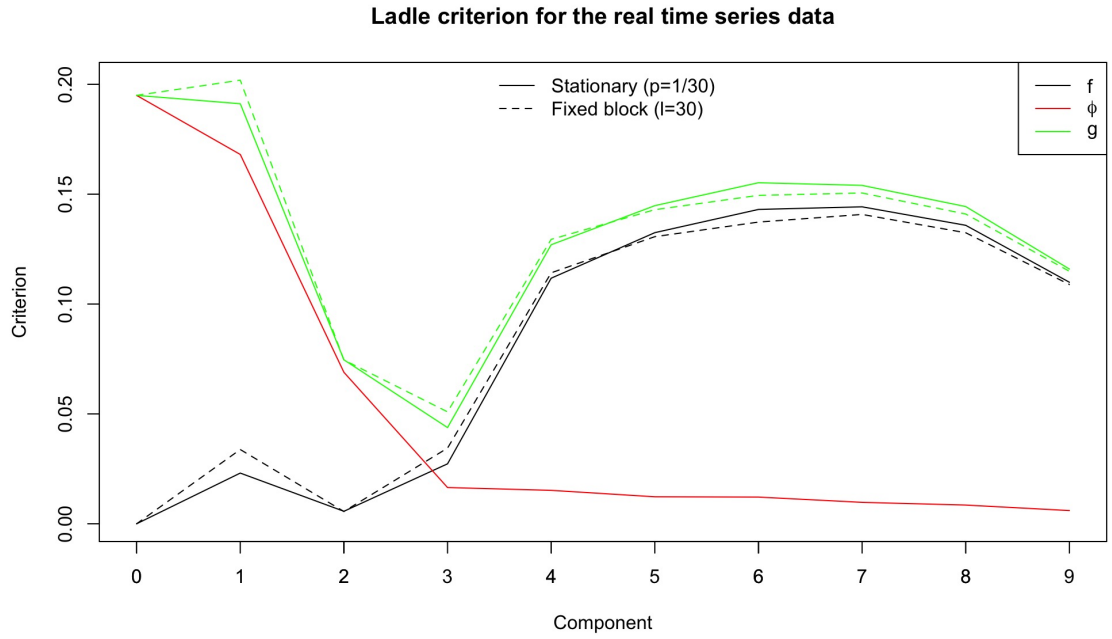


Figure 1.3: Comparison of the ladle criterion for the real time series data using stationary bootstrap ($p = 1/30$) and fixed block bootstrap ($l = 30$), with 500 bootstrap replications for each method. Solid lines represent the stationary bootstrap, while dashed lines represent the fixed block bootstrap.

The results suggest that the observed ten-dimensional time series can be represented by a lower-dimensional signal structure, with an estimated signal dimension of $\hat{q} = 3$. Under the SOBI framework, this indicates that three latent components exhibit relevant temporal dependence, while the remaining seven components can be interpreted as noise components, approximately behaving as white noise. Importantly, this does not imply that only three of the original observed variables contain useful information. Rather, the signal components correspond to linear combinations of the original variables, meaning that the temporal dependence may be distributed across several observed series. The estimated three-dimensional signal subspace therefore captures the main temporal structure of the multivariate dataset.

The agreement between the stationary and fixed block bootstrap methods, together with the stability of the estimate across a range of block lengths, provides evidence supporting this low-dimensional representation. Nevertheless, since the true signal dimension is unknown, the result should be interpreted as an estimate under the assumptions of the SOBI model rather than as a definitive identification of the underlying structure.

2. Bayesian mixture modelling with MCMC

We begin Task 2 by choosing suitable hyperparameter values. The means μ_1 and μ_2 follow the normal distribution with the mean of m_0 and the variance of s_0^2. If we are standardizing the data, we should have the mean of μ_1 and μ_2 be 0, so their values would also be centered around 0 like the standardized data. Because most of the standardized data will fall between -3 and 3, the prior standard deviation will be chosen as 5, which makes the variance 25. This way the prior is broad enough. So, $m_0 = 0$ and $s_0^2 = 25$.

The probability π follows the beta distribution with parameters a and b. To get a standard uniform prior, we set both $a = b = 1$. This way we start with no preference between $Z_i = 1$ and $Z_i = 2$. The variance σ^2 follows the inverse-gamma distribution with parameters α_0 and β_0. Because the variance of the standardized data will be 1, we pick α_0 and β_0, so that the mean $\frac{\beta_0}{\alpha_0 - 1}$ of the inverse-gamma distribution will be 1. So, $\alpha_0 = 2.5$ and $\beta_0 = 1.5$, because now $\frac{\beta_0}{\alpha_0 - 1} = \frac{1.5}{2.5 - 1} = 1$.

2.1 Latent-variable representation and Gibbs sampler

We will now derive the full conditional distributions required for the Gibbs sampler. Firstly, we derive the joint posterior that we will need for the subsequent derivations. We use the knowledge that the joint posterior distribution is proportional to the likelihood multiplied by the prior distributions of all parameters:

$$p(Z, \mu_1, \mu_2, \pi, \sigma^2 \mid y) \propto \left[\prod_{j=1}^{n} p(y_j \mid Z_j, \mu_1, \mu_2, \sigma^2) p(Z_j \mid \pi) \right] p(\mu_1) p(\mu_2) p(\pi) p(\sigma^2).$$

Now, we take a look at $P(Z_i = 1 \mid y_i, \mu_1, \mu_2, \pi, \sigma^2)$, and its complementary event $P(Z_i = 2 \mid y_i, \mu_1, \mu_2, \pi, \sigma^2)$. We get rid of all the constants from the joint posterior, when looking at the latent variable Z_i:

$$\begin{aligned} p(Z_i = 1 \mid y_i, \mu_1, \mu_2, \pi, \sigma^2) &\propto p(y_i \mid Z_i = 1, \mu_1, \mu_2, \sigma^2) p(Z_i = 1 \mid \pi) \\ &= \phi(y_i; \mu_1, \sigma^2)\pi. \end{aligned}$$

We can also derive the complementary event:

$$\begin{aligned} p(Z_i = 2 \mid y_i, \mu_1, \mu_2, \pi, \sigma^2) &\propto p(y_i \mid Z_i = 2, \mu_1, \mu_2, \sigma^2) p(Z_i = 2 \mid \pi) \\ &= \phi(y_i; \mu_2, \sigma^2)(1 - \pi). \end{aligned}$$

Let's assume that $k, t \in \{1, 2\}$ and $k \neq t$. Continuing, we will now derive the means μ_k:

$$\begin{aligned}
p(\mu_k \mid y, Z, \mu_t, \pi, \sigma^2) &\propto \left[\prod_{j=1}^{n} p(y_j \mid Z_j, \mu_k, \mu_t, \sigma^2)\right] p(\mu_k) \\
&\propto \left[\prod_{j:Z_j=k} p(y_j \mid \mu_k, \sigma^2)\right] p(\mu_k) \\
&\propto \left[\prod_{j:Z_j=k} \exp\left(-\frac{1}{2\sigma^2}(y_j - \mu_k)^2\right)\right] \exp\left(-\frac{1}{2s_0^2}(\mu_k - m_0)^2\right) \\
&= \exp\left(-\frac{1}{2\sigma^2}\sum_{j:Z_j=k}(y_j - \mu_k)^2\right) \exp\left(-\frac{1}{2s_0^2}(\mu_k - m_0)^2\right) \\
&= \exp\left(-\frac{1}{2}\left[\frac{\sum_{j:Z_j=k}(y_j - \mu_k)^2}{\sigma^2} + \frac{(\mu_k - m_0)^2}{s_0^2}\right]\right).
\end{aligned}$$

Let's assume that $n_k = \sum_{j:Z_j=k} 1$ is the number of observations allocated to component k, and the sum of those observations is $S_k = \sum_{j:Z_j=k} y_j$. Now we get:

$$\begin{aligned}
p(\mu_k \mid y, Z, \mu_t, \pi, \sigma^2) &\propto \exp\left(-\frac{1}{2}\left[\frac{\sum_{j:Z_j=k}(y_j^2 - 2y_j\mu_k + \mu_k^2)}{\sigma^2} + \frac{\mu_k^2 - 2\mu_k m_0 + m_0^2}{s_0^2}\right]\right) \\
&= \exp\left(-\frac{1}{2}\left[\frac{\sum_{j:Z_j=k} y_j^2 - 2\mu_k S_k + n_k\mu_k^2}{\sigma^2} + \frac{\mu_k^2 - 2\mu_k m_0 + m_0^2}{s_0^2}\right]\right) \\
&\propto \exp\left(-\frac{1}{2}\left[\frac{n_k\mu_k^2 - 2S_k\mu_k}{\sigma^2} + \frac{\mu_k^2 - 2m_0\mu_k}{s_0^2}\right]\right) \\
&= \exp\left(-\frac{1}{2}\left[\frac{n_k s_0^2\mu_k^2 - 2S_k s_0^2\mu_k + \sigma^2\mu_k^2 - 2m_0\sigma^2\mu_k}{s_0^2\sigma^2}\right]\right) \\
&= \exp\left(-\frac{1}{2}\left[\frac{(n_k s_0^2 + \sigma^2)\mu_k^2 - 2(S_k s_0^2 + m_0\sigma^2)\mu_k}{s_0^2\sigma^2}\right]\right) \\
&= \exp\left(-\frac{1}{2}\left[\mu_k^2\left(\frac{n_k}{\sigma^2} + \frac{1}{s_0^2}\right) - 2\mu_k\left(\frac{S_k}{\sigma^2} + \frac{m_0}{s_0^2}\right)\right]\right).
\end{aligned}$$

We notice that this matches the kernel of the normal distribution, so we can define the updated variance v_k^* and mean m_k^* as:

$$v_k^* = \left(\frac{n_k}{\sigma^2} + \frac{1}{s_0^2}\right)^{-1},$$

$$m_k^* = v_k^*\left(\frac{S_k}{\sigma^2} + \frac{m_0}{s_0^2}\right).$$

We now know that $\mu_k \mid y, Z, \mu_t, \pi, \sigma^2 \sim N(m_k^*, v_k^*)$. Now, we can derive the probability π:

$$\begin{aligned}
p(\pi \mid y, Z, \mu_1, \mu_2, \sigma^2) &\propto \left[\prod_{j=1}^{n} p(Z_j \mid \pi)\right] p(\pi) \\
&\propto \left[\prod_{j=1}^{n} \pi^{I(Z_j=1)}(1-\pi)^{I(Z_j=2)}\right] \pi^{a-1}(1-\pi)^{b-1} \\
&= \left(\pi^{\sum_{j:Z_j=1} 1}(1-\pi)^{\sum_{j:Z_j=2} 1}\right) \pi^{a-1}(1-\pi)^{b-1} \\
&= \pi^{n_1}(1-\pi)^{n_2}\pi^{a-1}(1-\pi)^{b-1} \\
&= \pi^{(a+n_1)-1}(1-\pi)^{(b+n_2)-1}.
\end{aligned}$$

We can now see that $\pi \mid y, Z, \mu_1, \mu_2, \sigma^2 \sim \text{Beta}(a+n_1, b+n_2)$. Finally, we can derive the variance σ^2:

$$\begin{aligned}
p(\sigma^2 \mid y, Z, \mu_1, \mu_2, \pi) &\propto \left[\prod_{j=1}^{n} p(y_j \mid Z_j, \mu_k, \mu_t, \sigma^2)\right] p(\sigma^2) \\
&\propto \left[\prod_{j=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2\sigma^2}(y_j - \mu_{Z_j})^2\right)\right] \frac{1}{(\sigma^2)^{\alpha_0+1}} \exp\left(-\frac{\beta_0}{\sigma^2}\right) \\
&\propto \left[(\sigma^2)^{-\frac{n}{2}} \exp\left(-\frac{1}{2\sigma^2}\sum_{j=1}^{n}(y_j - \mu_{Z_j})^2\right)\right] (\sigma^2)^{-(\alpha_0+1)} \exp\left(-\frac{\beta_0}{\sigma^2}\right) \\
&= (\sigma^2)^{-(\alpha_0+\frac{n}{2})-1} \exp\left(-\frac{1}{\sigma^2}\left[\beta_0 + \frac{1}{2}\sum_{j=1}^{n}(y_j - \mu_{Z_j})^2\right]\right).
\end{aligned}$$

We notice that this matches the kernel of the inverse-gamma distribution, so we can define the updated parameters α^* and β^* as:

$$\alpha^* = \alpha_0 + \frac{n}{2},$$

$$\beta^* = \beta_0 + \frac{1}{2}\sum_{j=1}^{n}(y_j - \mu_{Z_j})^2.$$

We now know that $\sigma^2 \mid y, Z, \mu_1, \mu_2, \pi \sim \text{Inverse-Gamma}(\alpha^*, \beta^*)$.

We can now implement the Gibbs sampler in R. Firstly, we initialize the parameters. Our algorithm supports inputted starting values, but it can also initialize automatically. The latent allocation variables Z_i get randomly assigned either component 1 or 2 with 50% probability for each observation y_i. The means μ_1 and μ_2 get set to the sample average of the data points initially allocated to the related component. If they happen to be identical, one is shifted down by 0.5 and the other up by 0.5. The probability π is set to the proportion of observations initially assigned to component 1. The proportion is bounded between 0.05 and 0.95 in the first iterations to prevent instability. The variance σ^2 is initialized using the overall sample variance of the entire dataset y.

For each iteration, we update the assignment Z_i for every observation y_i. We can use the derived conditional distribution to calculate the probabilities for belonging to either component. To ensure numerical stability and prevent underflow we perform the following on the log scale when computing the Gaussian densities:

$$\log(w_{1,i}) = \log(\pi) + \log\phi(y_i; \mu_1, \sigma^2),$$

$$\log(w_{2,i}) = \log(1-\pi) + \log \phi(y_i; \mu_2, \sigma^2).$$

We can now extract a normalized probability $p_{1,i} = p(Z_i = 1 \mid y_i, \mu_1, \mu_2, \pi, \sigma^2)$ using the logistic function on the difference of the log-weights:

$$p_{1,i} = \frac{1}{1 + \exp(-(\log(w_{1,i}) - \log(w_{2,i})))}.$$

Finally, a new allocation is sampled for each data point using $Z_i \sim \text{Bernoulli}(p_{1,i})$, where each success is assigned to component 1 and failure to component 2.

Next, we update the means μ_1 and μ_2. For each component k, so for 1 and 2, we calculate the summary statistics $n_k = \sum_{j:Z_j=k} 1$ and $S_k = \sum_{j:Z_j=k} y_j$ based on the just sampled allocations. The updated means are then drawn from the normal conditional distributions: $\mu_k \sim N(m_k^*, v_k^*)$. The probability π is updated by sampling from its beta conditional distribution: $\text{Beta}(a+n_1, b+n_2)$. Finally, the variance σ^2 is updated. We calculate the sum of squared residuals $\sum_{j=1}^{n}(y_j - \mu_{Z_j})^2$ using the current allocations and means from above. The new variance is then drawn from the inverse-gamma conditional distribution: $\sigma^2 \sim \text{Inverse-Gamma}(\alpha^*, \beta^*)$. This is then repeated for M iterations with the initial burn-in draws discarded to ensure accuracy in the representation of the joint posterior distribution.

To validate the Gibbs sampler, a pilot run was executed using synthetic data with 100 observations sampled from a normal distribution with a mean of -2 and a standard deviation of 1 to represent component 1, and another 100 samples from a normal distribution with a mean of 2 and a standard deviation of 1 to represent the second component. So, altogether there were 200 observations. They were picked to be far away from each other to ensure the two groups were distinct and easy to identify in the pilot run. The combined dataset is then standardized before being fed into the Gibbs sampler.

The pilot run used 6000 iterations with a 2000 burn-in. The length of the burn-in period was determined by testing with different lengths and seeing when the Markov chain seemed to converge. To reduce autocorrelation and save memory, the remaining 4000 draws were thinned by a factor of 4. This resulted in 1000 retained posterior samples. Finally, these retained draws were used to compute summary statistics for the parameters μ_1, μ_2, π and σ^2. The code stores the posterior mean, standard deviation, median, and the 2.5% and 97.5% empirical quantiles, which construct the 95% credible intervals.

2.2 Simulation study and computational behaviour

To make sure that the Gibbs sampler's implementation is valid and robust, we conducted a simulation study for two situations. In both cases, $N = 300$ observations were generated from a two-component Gaussian mixture model with equal mixture weights of 50%, and a variance of 1. As we can see from Figure 2.1, in the first setting, the two components of the mixture are clearly separated. This was achieved by setting $\mu_1 = -2$ and $\mu_2 = 2$ so that the data is highly distinct. In the second setting, the two components overlap substantially, which was achieved by setting $\mu_1 = -0.75$ and $\mu_2 = 0.75$.

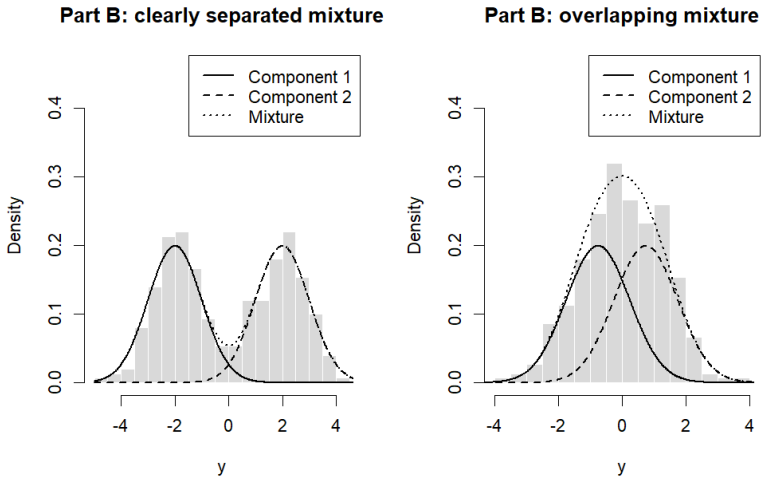


Figure 2.1: The histograms from the simulated observations show how the two scenarios differ with our chosen parameters.

These two settings differ in computational difficulty. In the clearly separated scenario, the data strongly informs the latent variables Z_i. The sampler can assign observations to the correct components with high confidence. However in the substantially overlapping scenario, many observations fall into the same shared region, making the source of those data points ambiguous. This uncertainty makes the updated parameters also more uncertain, forcing the Markov chain to move quite slowly through the parameter space, and often resulting in higher autocorrelation, lower effective sample sizes, and a high risk of label switching.

To assess convergence and sensitivity to starting values, three Markov chains were run for each scenario from dispersed initial values. Each chain ran for 12000 iterations. The first 4000 were discarded as burn-in, and the remaining were thinned by a factor of 2, resulting in 4000 draws for each chain. Since the model does not know which group is which, we reordered the results afterward to keep them consistent. Parameters were ordered into a lower-mean component μ_{low} and an upper-mean component μ_{high} with their weights and allocations. This allowed for good summaries without using artificial constraints on the Gibbs sampler itself.

For the clearly separated mixture, the Gibbs sampler was computationally highly efficient. As detailed in Table 2.1, the $\hat{R}$ values were approximately 1 for all parameters, suggesting good agreement between the three chains and providing evidence consistent with convergence. The trace plots in Figure 2.2 showed quick mixing with quite stable trajectories, and the autocorrelation function (ACF) in Figure 2.3 dropped to zero almost instantly. The approximated effective sample sizes (ESS) were all massive, over 10600 out of the 12000 stored draws. Furthermore, a half-run stability check showed negligible differences, less than 0.004, between the first and second halves of the retained draws. The true data-generating parameters were successfully recovered within the 95% credible intervals.

Table 2.1: MCMC diagnostics for the clearly separated mixture based on 12000 stored draws.

Parameter	Posterior SD	ESS Total	Approx. MCSE	$\hat{R}$
μ_{low}	0.083	10697	0.001	1.000
μ_{high}	0.084	10837	0.001	1.000
π_{low}	0.030	10994	< 0.001	1.000
σ^2	0.078	11091	< 0.001	1.000

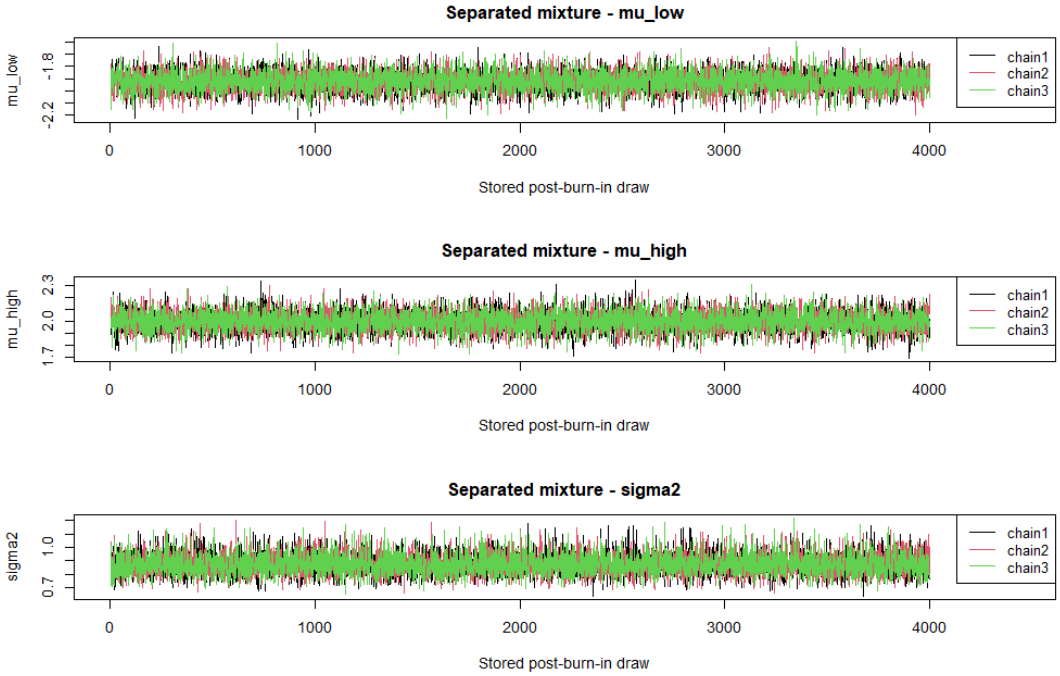


Figure 2.2: The trace plots for the clearly separated mixture show quick mixing, indicating fast and stable convergence.

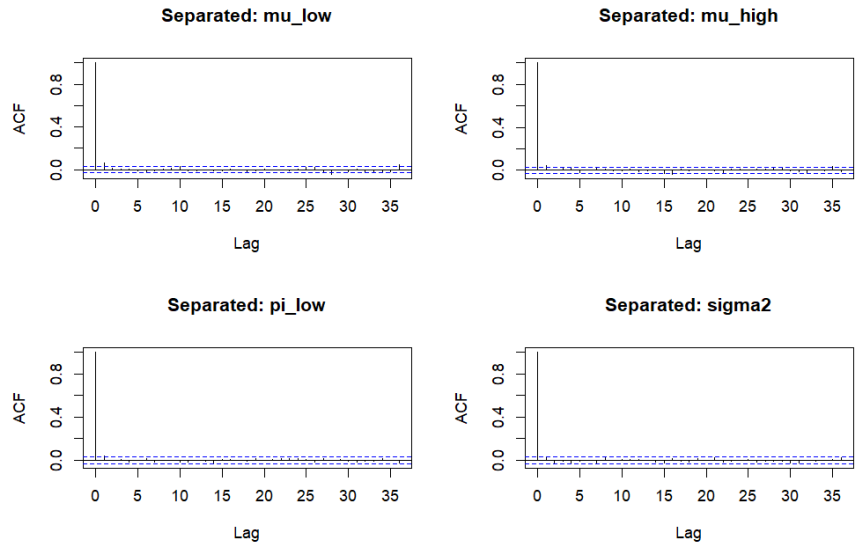


Figure 2.3: The autocorrelation function (ACF) drops to zero almost instantly with the clearly separated mixture.

Table 2.2 shows the posterior mean for μ_{low} was -1.922, the 95% credible interval $[-2.085, -1.759]$, and the true parameter was -2, the variance was 0.872, the 95% credible interval for it $[0.734, 1.039]$, and the true parameter was 1. The latent variables were classified with a 97% accuracy rate and extremely low allocation uncertainty of 0.040, as the posterior densities in Figure 2.4 show.

Table 2.2: Posterior summaries for the clearly separated mixture. The true values were $\mu_{\text{low}} = -2$, $\mu_{\text{high}} = 2$, $\pi_{\text{low}} = 0.5$, and $\sigma^2 = 1$.

Parameter	Mean	SD	2.5%	Median	97.5%
μ_{low}	-1.922	0.083	-2.085	-1.922	-1.759
μ_{high}	2.001	0.084	1.838	2.001	2.164
π_{low}	0.507	0.030	0.447	0.507	0.566
σ^2	0.872	0.078	0.734	0.867	1.039

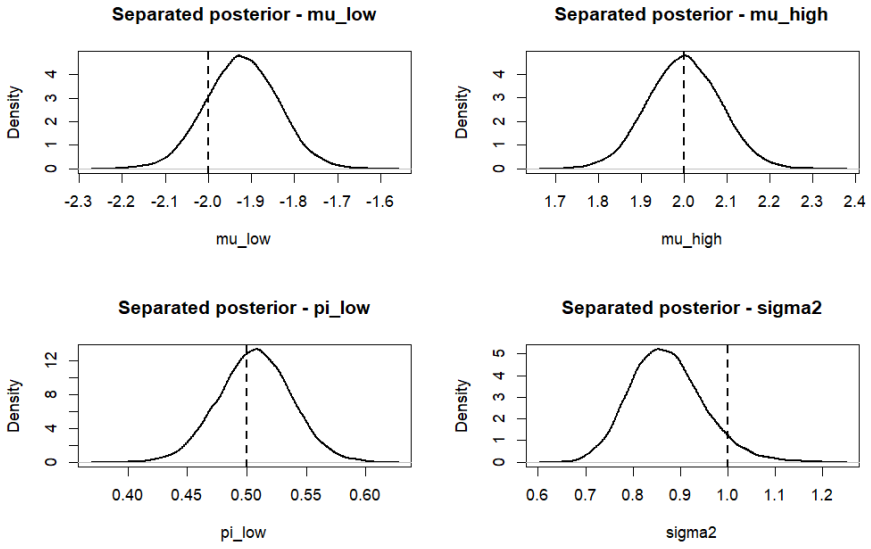


Figure 2.4: In the clearly separated scenario, the posterior densities for the parameters are either spot on or close enough. The σ^2 posterior has its peak visually the furthest away from the true value.

For the substantially overlapping mixture, the Gibbs sampler showed its limitations. Table 2.3 shows the $\hat{R}$ values stayed close to one, which was reassuring, but the autocorrelation function (ACF) decayed very slowly as is shown in Figure 2.6, indicating strong serial dependence inside the chain. The effective sample sizes (ESS) were very small, around $350-750$ out of the 12000 total draws. The half-run stability check also showed substantial instability in Figure 2.5, with mean differences between halves reaching up to 0.193 for μ_{low} in the third chain. While the true values fell within the 95% credible intervals, the intervals themselves were too wide and the posterior distributions were irregular.

Table 2.3: MCMC diagnostics for the substantially overlapping mixture based on 12000 stored draws.

Parameter	Posterior SD	ESS Total	Approx. MCSE	$\hat{R}$
μ_{low}	0.679	714	0.025	1.001
μ_{high}	0.509	1867	0.012	1.001
π_{low}	0.258	633	0.010	1.003
σ^2	0.257	352	0.014	1.002

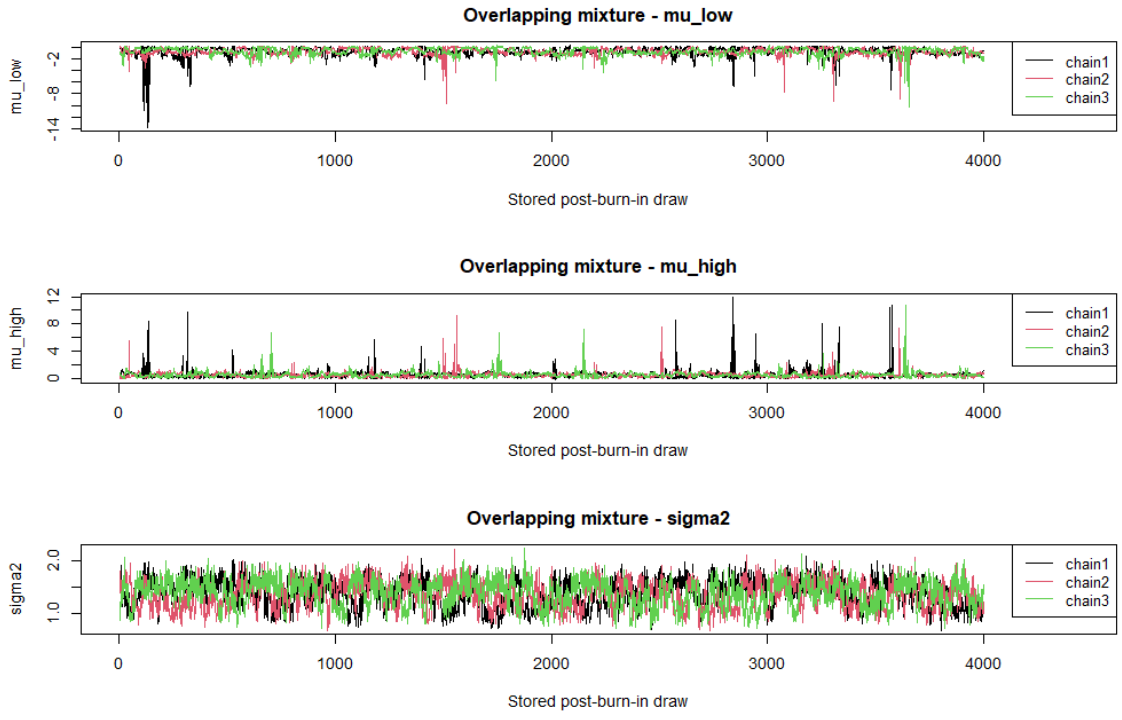


Figure 2.5: The trace plots for the substantially overlapping mixture show some instability and serial dependence.

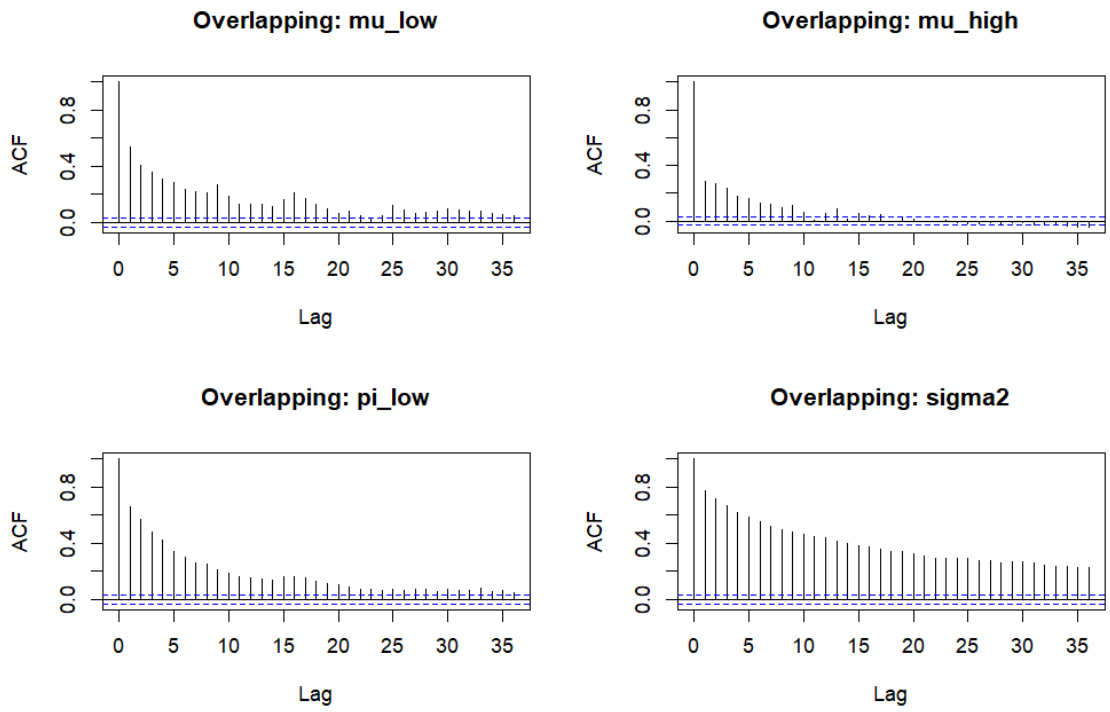


Figure 2.6: The autocorrelation function (ACF) decays very slowly with the substantially overlapping mixture.

As shown in Table 2.4, the posterior mean for μ_{low} was -0.741, the 95% credible interval of $[-2.043, 0.077]$, and the true parameter of -0.75. The structural ambiguity degraded the latent variable allocation recovery, dropping classification accuracy to 72.7% and making the mean allocation uncertainty jump to 0.667 as is shown in Table 2.5. The posterior densities are correspondingly wide and irregular, which can be seen in Figure 2.7.

Table 2.4: Posterior summaries for the substantially overlapping mixture. The true values were $\mu_{\text{low}} = -0.75$, $\mu_{\text{high}} = 0.75$, $\pi_{\text{low}} = 0.5$, and $\sigma^2 = 1$.

Parameter	Mean	SD	2.5%	Median	97.5%
μ_{low}	-0.741	0.679	-2.043	-0.717	0.077
μ_{high}	0.473	0.509	0.005	0.406	1.270
π_{low}	0.390	0.258	0.019	0.345	0.972
σ^2	1.361	0.257	0.867	1.385	1.805

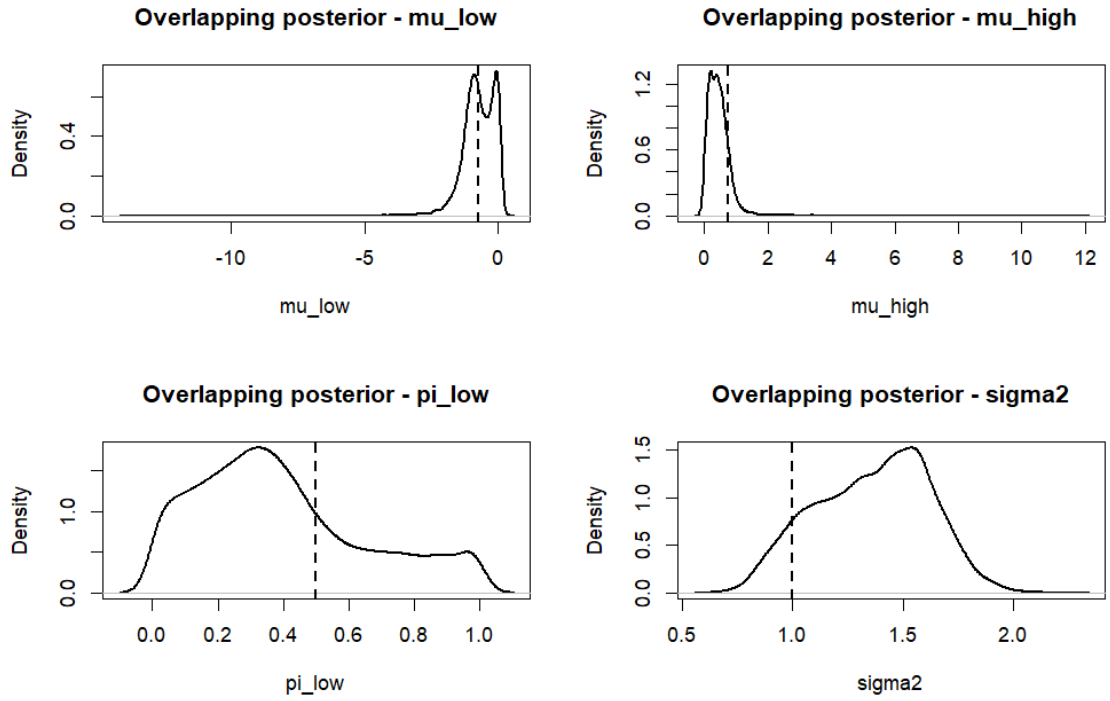


Figure 2.7: In the substantially overlapping scenario, the posterior densities for the parameters are mostly uncertain and excessively wide, and some have multiple peaks.

Table 2.5: Latent-allocation recovery accuracy across scenarios with $N = 300$.

Scenario	Classification accuracy	Mean allocation uncertainty
Separated	97.0%	0.040
Overlapping	72.7%	0.667

The diagnostics confirmed that 12000 iterations with 4000 burn-in were adequate for the clearly separated mixture, yielding nearly independent samples and highly precise inference. However, for the substantially overlapping mixture, the low ESS, high autocorrelation, and half-run instability showed that the sampler was working inefficiently. While the $\hat{R}$ might have falsely suggested convergence, the complete diagnostics indicated that a significantly longer run length or a better sampling algorithm would be required to achieve reliable posterior inference in the overlapping scenario.

2.3 Label switching and identifiability

While label switching usually manifests as the component traces trading places, the raw trace plots for μ_1 and μ_2 in the overlapping scenario are squashed as is shown in Figure 2.8, because of the sampler's instability producing extreme excursions. This makes it a lot harder to see if there has been label switching. However, in the raw probability π there is clear label switching visible. As the Markov chains exchange identities of the two components, the traces for π oscillate back and forth across the 0.5 threshold in wide waves throughout the simulation.

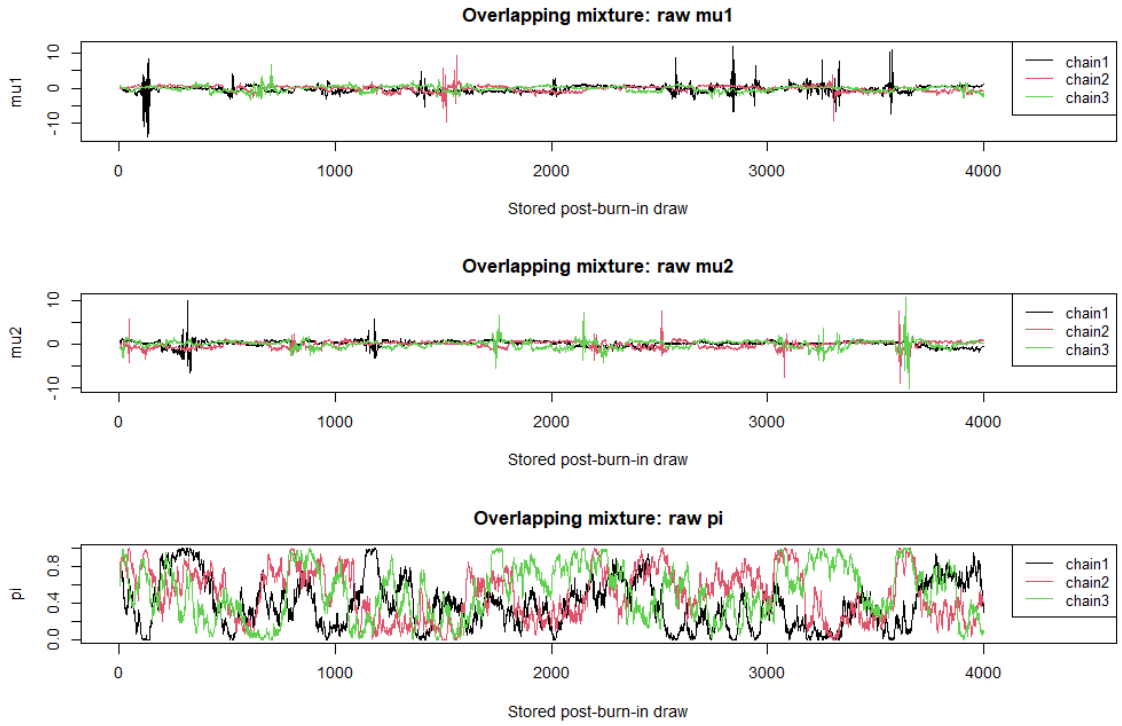


Figure 2.8: The outliers squash the μ_1 and μ_2 plots, but the trace plot for π shows how often the labels can switch.

Calculating ordinary posterior summaries from the raw chains where label switching occurs produces misleading parameter estimates. Because the sampler switches between the two components, a mean of the draws only averages the mirrored waves into the middle. In the substantially overlapping mixture, the raw diagnostic output in Table 2.6 shows that the sampler spent roughly half of its iterations in each configuration, causing the raw posterior means to collapse towards zero in Table 2.7. Therefore, the raw posterior means for both μ_1 and μ_2 collapsed to approximately -0.182 and -0.087. These values describe a middle component that does not exist rather than accurately reflecting the true data-generating parameters -0.75 and 0.75.

Table 2.6: Raw label-order diagnostics showing the proportion of draws where $\mu_1 < \mu_2$ and the number of times the labels switched within the 4000 draws per chain.

Scenario	Chain	Proportion ($\mu_1 < \mu_2$)	Switches
Separated	Chain 1	1.000	0
Separated	Chain 2	0.000	0
Separated	Chain 3	1.000	0
Overlapping	Chain 1	0.605	305
Overlapping	Chain 2	0.478	244
Overlapping	Chain 3	0.513	239

Table 2.7: Raw posterior summaries for the overlapping mixture, demonstrating how the means of μ_1 and μ_2 collapse toward zero due to label switching.

Parameter	Mean	SD	2.5%	Median	97.5%
μ_1	-0.182	0.902	-1.756	-0.016	1.076
μ_2	-0.087	0.799	-1.664	0.079	0.959
π	0.467	0.278	0.019	0.442	0.976
σ^2	1.361	0.257	0.867	1.385	1.805

Even when the identities of the components switch freely, some global properties of the mixture remain fixed and give meaningful inference without assigned labels. Absolute **separation** $|\mu_1 - \mu_2|$ captures the distance between components regardless of the labels, overall **mixture mean** $\pi\mu_1+(1-\pi)\mu_2$ remains at the center of the data, and the variance σ^2 is shared equally across all observations regardless of their assigned label, so the raw invariant posterior summaries remain valid as can be seen in Table 2.8.

Table 2.8: Posterior summaries for the quantities, which remain meaningful regardless of label switching.

Scenario	Quantity	Mean	SD	2.5%	Median	97.5%
Separated	Separation	3.923	0.110	3.706	3.923	4.141
Separated	Mixture mean	0.013	0.126	−0.235	0.012	0.261
Overlapping	Separation	1.214	0.770	0.077	1.257	2.418
Overlapping	Mixture mean	0.043	0.074	−0.103	0.044	0.187

To obtain interpretable component-specific summaries, some restriction must be put in place. Enforcing a parameter restriction $\mu_1 < \mu_2$ within the Gibbs sampler's transition steps can artificially cut off the target distribution, distort the posterior's shape, or restrict the Markov chain's ability to mix especially when components overlap.

A better approach is the post-processing strategy. By allowing the sampler to freely explore the possibilities, the Markov chains mix efficiently. Identifiability is then restored by ordering the retained draws into μ_{low} and μ_{high} after sampling, and relabeling π and Z_i for each iteration based on that sorted order. In the ordered outputs we can see that this post-processing successfully reconstructs the distinct component parameters without interfering with the underlying computational system.

2.4 Application to real data

To evaluate the practical utility of the two-component Gaussian mixture model, it was applied to the eruption durations of the Old Faithful geyser using the built-in `faithful` dataset in R. The dataset consists of $N = 272$ observations. An initial visual check of the raw data in Figure 2.9 revealed a clear bimodal distribution, making it a great candidate for mixture modeling.

The data was standardized before being fed into the Gibbs sampler to ensure computational stability. Three Markov chains were run, and each of them was 12000 iterations long before removing the 4000 burn-in and thinning by a factor of 2, making the amount of analyzed data points 4000 per chain. To account for the label-switching issue discussed in section 2.3, the retained draws were post-processed by ordering the component means into μ_{low} and μ_{high}. Finally, the posterior draws were transformed back to the original scale.

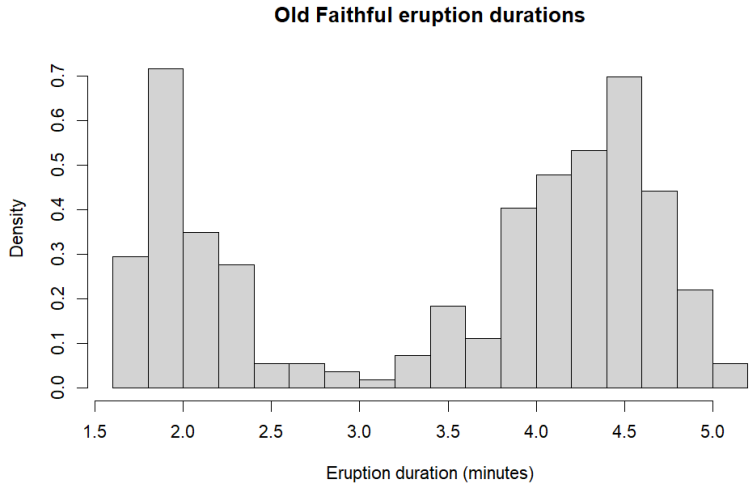


Figure 2.9: A histogram of the observed Old Faithful eruption durations showing a distinct bimodal structure.

The MCMC performance on the real data was exceptionally strong. The trace plots in Figure 2.10 showed stable trajectories for all of the three chains. As shown in Table 2.9, the classical $\hat{R}$ values were exactly 1 for all parameters, and the approximate effective sample sizes (ESS) were large, all exceeding 11000 draws out of the 12000 total retained draws. This shows how the chains mixed rapidly, and effectively explored the posterior distribution.

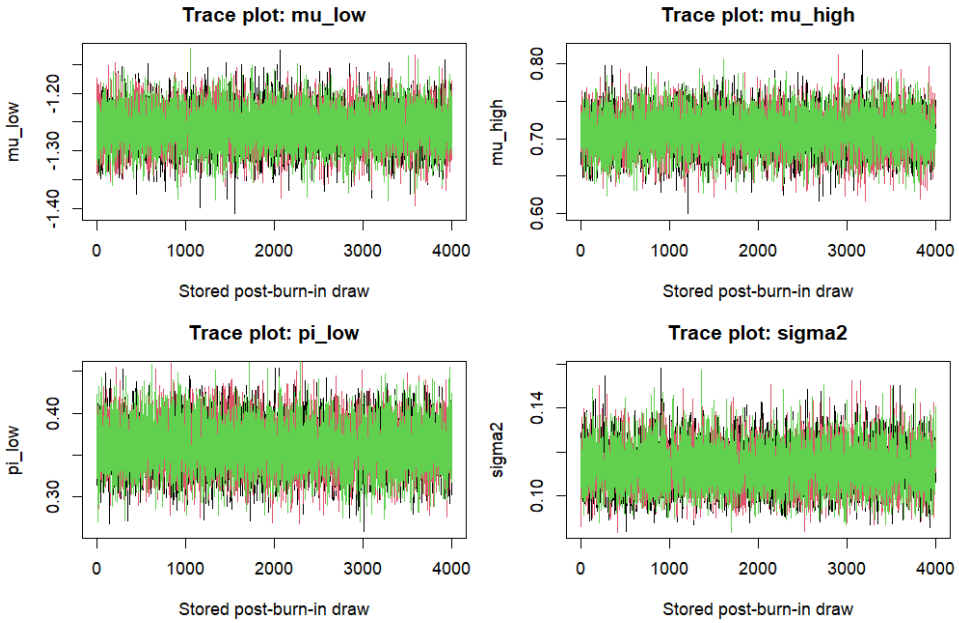


Figure 2.10: Trace plots for the parameters show highly stable mixing across all three chains for the Old Faithful data.

Table 2.9: MCMC diagnostics for the Old Faithful dataset based on 12000 stored draws across three chains.

Parameter	ESS Total	$\hat{R}$
μ_{low}	11888	1.000
μ_{high}	11181	1.000
π_{low}	11509	1.000
σ^2	11521	1.000

The posterior summaries in Table 2.10 reveal two different eruption profiles, some of the eruptions were shorter and some were longer. The short eruption component has a posterior mean duration of 2.05 minutes, accounting for approximately 36.2% of the observations. The long eruption component has a posterior mean duration of 4.3 minutes. The variance across these components is estimated at 0.147 squared minutes.

Table 2.10: Posterior summaries for the Old Faithful eruption durations, accounting for label-switching via post-processing.

Parameter	Mean	SD	2.5%	Median	97.5%
μ_{low} (minutes)	2.048	0.040	1.971	2.047	2.127
μ_{high} (minutes)	4.297	0.029	4.241	4.297	4.356
π_{low}	0.362	0.029	0.306	0.363	0.420
σ^2 (minutes^2)	0.147	0.013	0.125	0.146	0.174

Visually the two-component Gaussian mixture provides a highly useful description of the data. Overlaying the posterior mean fitted mixture density onto the histogram in Figure 2.11 shows that the model accurately captures the locations, spread, and relative weighting of both peaks.

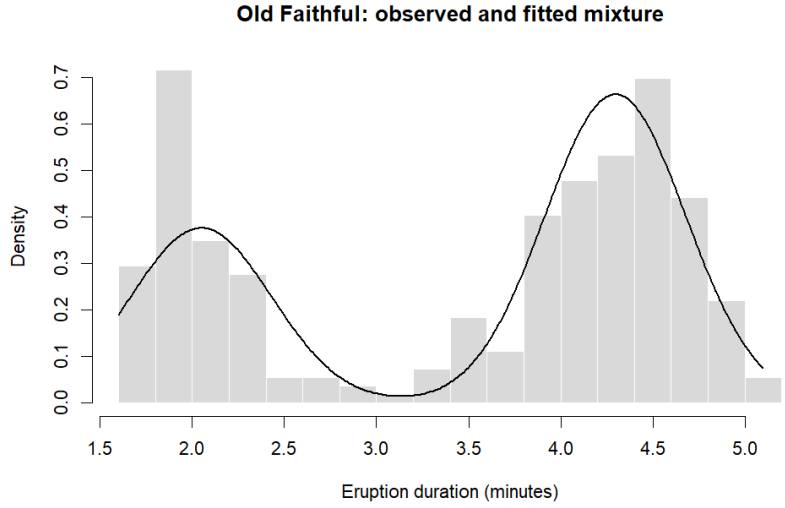


Figure 2.11: The two-component Gaussian mixture accurately captures the bimodal distribution of the eruption durations.

This strong model fit is further supported by analyzing the posterior label allocation uncertainty for singular observations. As illustrated in Figure 2.12, the majority of the 272 eruptions sit at an allocation uncertainty of almost zero, meaning the model can confidently assign them to either the short or long component. Only a single observation exhibits a label allocation uncertainty above 0.25, representing a rare eruption duration that falls in the sparse region between the two peaks.

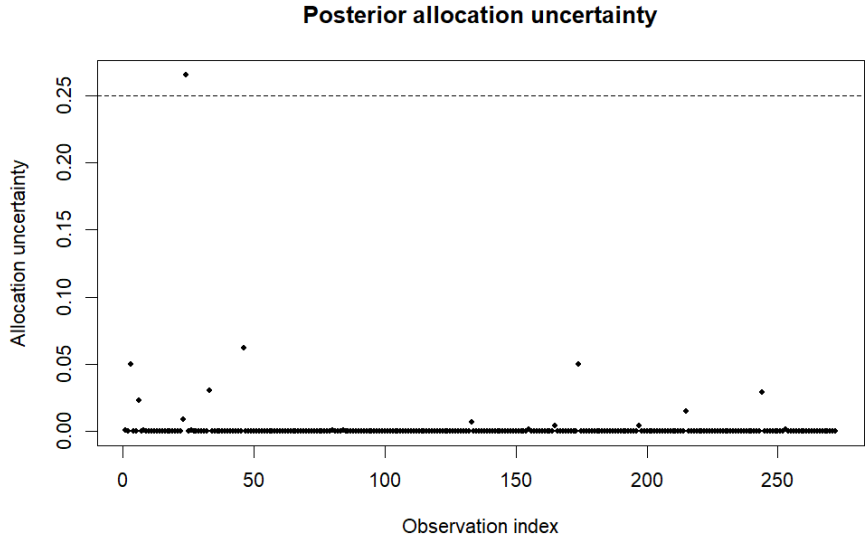


Figure 2.12: Posterior allocation uncertainty for the observed data. The vast majority of observations are assigned to a component with almost perfect certainty.

In conclusion, the two-component Gaussian mixture model is highly suitable for this dataset, providing precise analysis and great visual fit. However, these components must be interpreted cautiously. While the model reliably clusters the data into short and long categories, this successful description does not strictly imply that the geyser is driven by two physically distinct underlying populations or mechanisms.